

# Cognexis: A Compute-Adaptive Recurrent-Depth Language Model Architecture

Marlon Hanks  
Independent Researcher  
m@dust.11c

Open Specification — April 2026

## Abstract

Cognexis is a family of decoder-only large language models that separate reasoning depth from model size via recursive computation. Instead of scaling the capacity of the network with increasing depths of layers of novel transformer modules, the Cognexis model leverages a recursive reasoning core and runs it iteratively. In other words, while allowing to allocate more compute to challenging inputs, the model keeps the size of its weights invariant. This paper presents a full-fledged architecture, consisting of a tokenizer, rotary positional embedding, multi-layered Prelude–Recurrent–Coda model body, an adaptive looping scheduler with a learned value head, as well as an end-to-end training and evaluation pipeline. Novel metrics are introduced to measure how effectively the reasoning depth benefits performance: the Depth Efficiency Index (DEI), Loop Saturation Point (LSP), Overthinking Threshold (OT), and Depth Gain Ratio (DGR). The model architecture can accommodate different kinds of iteration strategies: fixed, sequence-dependent, and token-dependent. Architectures with sizes ranging from 8 billion to 1.28 trillion parameters are considered.

## 1 Introduction

### 1.1 Motivation

Large-scale language models of today have demonstrated an impressive level of competency through their scale-up: many more layers, larger dimensions of their hidden representations, and increased size of parameters [1]. Progress in large language models has largely followed a scaling paradigm in which increases in layer depth, parameter count, and training data lead to consistent improvements in capability [1]. While effective, this approach treats performance gains as a direct function of resource expansion rather than architectural efficiency. Such models are capable of producing essays, proving mathematical statements, and writing software programs. However, there is a downside of such design that seems trivial but is very important nonetheless. Any input sentence has to be processed entirely by the whole architecture. For example, a system with 96 layers will use all of them to predict the next word in “The cat sat on the \_\_\_\_.” The amount of computational resources devoted to trivial tasks is equal to those needed for difficult tasks. Reasoning depth is tightly coupled with memory consumption [1].

There are at least two forms of inefficiency inherent in this architecture. The first form is that the inference cost will be higher for simpler inputs than necessary. The second inefficiency is that the effective depth of the neural network is constrained at the point of design, in that the model will never reason to a greater depth than the total number of layers it is composed of [1]. One

alternative strategy is that of Recurrent-Depth Transformers (RDTs). Rather than designing a neural net made up of tens of layers each with their own unique parameters, the RDT repeats a single recurrent layer. The fundamental realization is that the depth of the reasoning process need not depend on the complexity of the model [1].

The proposed framework does not exist in theory alone but has been validated in practice with recent studies into recurrent-depth language models, which have established their viability as a language model in practice. In particular, the study on the Huginn-3.5B model (Geiping et al., 2025) has shown that such a model has high potential for producing significant improvements in reasoning tasks, especially mathematics and programming, through test-time compute scaling of latent space. On the other hand, the subsequent analysis by Lu et al. (2025) has found little support for the claim that such models generate an interpretable latent chain of thoughts, with diminishing marginal returns on iterations [1].

What has not been done in the research so far is to provide a fully built model. The recurrent-depth models that have been built before were proof-of-concept versions that worked but left plenty of room for improvements in terms of engineering, including the integration with tokenizer, adaptation of scheduling algorithm, safety management, distributed training, and evaluation methods [1]. Cognexis addresses that problem by building a full-fledged LLM based on the recurrent depth concept, including a compute-adaptive interface [1].

## 1.2 Contributions

This work makes the following contributions:

1. **Compute-Adaptive Architecture.** This refers to a decoder-only network whose reasoning depth depends on the recurrence steps used during decoding. The model utilizes a three-part body, namely Prelude, Recurrent Core, and Coda. The Prelude consists of standard transformer layers that generate the first representation, the Recurrent Core processes them recursively, while the Coda integrates the output before passing it to the next layer [1].
2. **Complete LLM Stack.** Specifications for the entire stack – tokenization including special tokens, embedding layers using rotary position encoding, weight-sharing language modeling head, training pipeline with curriculum loops, and inference pipeline with KV-cache management. Model configurations at parameter sizes of 8 billion, 64 billion, 256 billion, and 1.28 trillion parameters [1].
3. **Adaptive Loop Scheduler.** Implement an inference scheduler that determines the number of recurrent loops required based on the anticipated marginal improvement in accuracy gained per computational resource. This scheduler can operate in fixed, rule-driven, value-driven, or hybrid scheduling methods, with hard safety constraints that cannot be bypassed [1].
4. **Novel Evaluation Metrics.** Inclusion of four novel evaluation criteria: Depth Efficiency Index (DEI), Loop Saturation Point (LSP), Overthinking Threshold (OT), and Depth Gain Ratio (DGR). These measures are explicitly created for assessing the effectiveness of recurrent depth on performance [1].
5. **Build-Ready Specification.** Whereas previous work describes research findings, this paper offers a full-fledged engineering specification, including schemas for configuration, pseudocode, memory layout, and procedures for distributed training and inference. Specifications are formulated not as research conclusions but as implementation agreements [1].

## 2 Background and Related Work

### 2.1 Transformer Scaling

The Transformer architecture (Vaswani et al., 2017) now serves as the foundation for most modern large language models because its attention mechanism allows each token to incorporate information from other positions in the context window. This enables the model to capture long-range dependencies more effectively than earlier recurrent approaches [1].

A common path to improving transformer performance has been to increase scale by adding more layers, widening hidden representations, and expanding parameter counts [1]. Although this strategy has produced major gains, it also increases memory usage, inference latency, and training cost. Each added layer introduces a new set of weights that must be stored, loaded, and applied sequentially [1].

As a result, brute-force scaling faces diminishing returns: performance continues to improve, but each improvement requires disproportionately greater computational investment [1]. This motivates architectures that increase effective reasoning depth through repeated computation rather than through proportional growth in stored parameters [1].

### 2.2 Recurrent Architectures and Deep Equilibrium Models

The idea of applying the same parameters across multiple computational steps predates modern transformers. Traditional recurrent neural networks reuse weight matrices at each step, allowing sequences of varying length to be processed without increasing the number of learned parameters [1]. Although RNNs are no longer dominant in language modeling, their core principle of weight reuse remains relevant for architectures that seek greater effective depth without larger parameter footprints [1].

Deep Equilibrium Models Bai et al. (2019) extend this principle by treating depth as an iterative process that converges toward a fixed point [1]. Instead of stacking many independently parameterized layers, a DEQ repeatedly applies a transformation until the hidden state stabilizes, allowing very deep computation to be approximated with a compact set of weights [1].

The Universal Transformer (Dehghani et al., 2019) adapts related ideas to transformer-style models by reusing parameters across repeated computational steps and incorporating adaptive halting [1]. However, repeated application of a shared transformation can also amplify instability, causing divergence, collapse, or oscillatory behavior if the dynamics are not carefully controlled [1].

### 2.3 Adaptive Computation Time

Adaptive Computation Time (ACT) (Graves, 2016) provides a mechanism for allowing a model to vary the amount of computation used for different inputs [1]. Rather than executing a fixed number of recurrent steps, the model evaluates at each step whether further processing is warranted, using a learned halting signal to guide this decision [1].

ACT also introduces a ponder cost, which discourages unnecessary computation by penalizing excessive processing steps [1]. This creates an incentive for the model to solve simpler cases quickly while reserving additional computation for inputs that require more refinement [1]. Because the mechanism is differentiable, it can be optimized alongside the main model objective [1].

In Cognexis, this principle is extended through a scheduler that does not rely only on accumulated halting probability [1]. Instead, the system includes a learned value head that estimates the expected benefit of another recurrent iteration, combined with hybrid decision rules and explicit safety constraints [1].

## 2.4 Contemporary Recurrent-Depth Language Models

Recent empirical support for recurrent-depth transformers as practical language models is provided by Geiping et al. (2025) through the Huginn-3.5B model. This work demonstrates that iterative refinement within a shared recurrent block can yield meaningful performance improvements, particularly on reasoning-intensive tasks such as mathematics and code generation [1]. By increasing the number of recurrent iterations at inference time, the model effectively scales its computational depth without increasing parameter count [1]. These results indicate that recurrent depth is not solely a mechanism for parameter efficiency, but can extend the model’s capacity for problems that benefit from sustained internal deliberation [1].

At the same time, subsequent analysis by Lu et al. (2025) introduces a more cautious perspective. Their investigation of Huginn-3.5B finds limited evidence that the recurrent iterations correspond to interpretable latent chains of reasoning. Moreover, the marginal gains from increasing iteration depth appear modest when compared to approaches that explicitly externalize intermediate reasoning steps, such as chain-of-thought prompting [1]. This suggests that while recurrent computation contributes to improved performance, its effects may be constrained and not fully aligned with transparent reasoning processes [1].

Taken together, these findings position recurrent-depth architectures as a promising but not yet fully understood direction. The approach offers measurable benefits, yet raises open questions about interpretability, efficiency, and scaling behavior [1]. Cognexis builds on this foundation by treating recurrent depth not as an isolated technique, but as part of a complete system design that addresses the architectural, training, evaluation, and deployment considerations left open by earlier implementations [1].

## 3 Architecture

### 3.1 Model Overview

Cognexis is a decoder-only language model that operates over token sequences and generates outputs autoregressively [1]. Its defining characteristic is a three-stage architecture that separates the forward pass into distinct phases: representation formation, iterative refinement, and output integration [1]. This separation is intentional rather than cosmetic, reflecting the view that these stages serve fundamentally different roles and are best supported by different architectural constraints [1].

The three stages are:

1. **Prelude:** A stack of standard transformer blocks with independent parameters, applied to raw token embeddings to produce well-formed initial representations. Its role is to stabilize the hidden state and embed it with sufficient contextual information before the recurrent refinement stage begins [1].
2. **Recurrent Core:** A single transformer block with shared parameters that is applied repeatedly to the hidden state for a configurable number of iterations. Each pass incrementally refines the representation, effectively increasing computational depth without introducing additional parameters. The number of iterations – referred to as the loop count – serves as the primary control over the model’s reasoning depth [1].
3. **Coda:** A stack of standard transformer blocks that operate on the refined representations produced by the recurrent stage. Its function is to integrate information across the sequence and transform the iterated hidden state into a form suitable for the final language modeling head [1].

The complete data flow through the model is:

```
input_text -> tokenizer.encode -> token_ids -> embedding(token_ids) ->
prelude(hidden, causal_mask) -> recurrent_core(hidden, scheduler, loop_budget) ->
coda(hidden, causal_mask) -> lm_head(hidden) -> logits
```

For a fixed loop count  $N$ , the effective depth of the model – the total number of transformer block applications – is given by:

$$D_{\text{eff}} = L_p + (N \times L_r) + L_c \quad (1)$$

where  $L_p$  is the number of Prelude blocks,  $L_r$  is the number of blocks in the recurrent core (baseline: 1),  $N$  is the loop count, and  $L_c$  is the number of Coda blocks [1]. For the Cognexis-8B configuration with 8 Prelude blocks, 1 recurrent block, 8 Coda blocks, and 12 loops, this yields an effective depth of 28 – comparable to a 28-layer standard transformer but with the parameter footprint of a 17-layer model [1]. Crucially, increasing  $N$  increases effective depth without adding any unique parameters [1].

### 3.2 Tokenization and Embeddings

Cognexis employs a SentencePiece tokenizer with a subword vocabulary of approximately 50,000 tokens [1]. The vocabulary includes standard special tokens – such as beginning-of-sequence (BOS), end-of-sequence (EOS), and padding – as well as dedicated markers for system, user, and assistant turns, enabling native handling of multi-turn conversational inputs [1].

Token embeddings map each token ID to a learned dense vector of dimension  $d_{\text{model}}$ . Rotary Position Encodings (RoPE) (Su et al., 2024) are applied to query and key vectors within every attention layer, including those in the recurrent core [1]. RoPE encodes positional information through rotation matrices applied in the complex plane, providing relative position sensitivity without learned position embeddings [1]. Importantly, RoPE’s design supports extrapolation to sequence lengths beyond those seen during training, a valuable property for a model whose effective depth varies at inference time [1].

### 3.3 The Prelude

The Prelude consists of  $L_p$  standard transformer blocks – typically between 6 and 12 in the 8B configuration, and scaling up to 16 in larger models such as the 1.28T variant [1]. Each block follows a pre-norm design, incorporating RMS LayerNorm, multi-head self-attention with Grouped Query Attention (GQA) (Ainslie et al., 2023), and a feed-forward network using SwiGLU activation (Shazeer, 2020) followed by a linear projection, with residual connections applied after each sub-layer [1].

Within GQA, query heads are partitioned into groups, where each group shares a common set of key and value projections [1]. This design reduces the memory footprint of the key–value cache during inference while retaining most of the expressive capacity associated with full multi-head attention [1].

The Prelude is responsible for constructing well-formed, contextually grounded representations before the recurrent stage begins [1]. Without this stage, the recurrent core would operate on relatively unstructured embeddings, requiring early iterations to focus on basic feature extraction rather than higher-level refinement [1]. By the time the hidden state enters the recurrent core, it already captures local syntax, entity relationships, and broader contextual semantics, providing a stable and information-rich starting point for iterative processing [1].

### 3.4 The Recurrent Core

The Recurrent Core forms the central component of the Cognexis architecture. It consists of a single transformer-style block  $\mathcal{R}$  with shared parameters, applied iteratively to the hidden state for  $N$  steps [1]. At iteration  $t$ , the hidden state  $h_t$  is updated according to:

$$h_{t+1} = \mathcal{R}(h_t) \quad (2)$$

Here,  $\mathcal{R}(\cdot)$  denotes a standard transformer-style transformation composed of RMS layer normalization, multi-head self-attention with Grouped Query Attention (GQA), a SwiGLU-based feed-forward network, and residual connections [1]. The same set of weight matrices is reused at every iteration, with no iteration-specific parameters in the base design [1]. This parameter sharing enables the model to increase its effective computational depth through repeated application of  $\mathcal{R}$ , without increasing the total number of learned parameters [1].

Naive iteration of a shared transformation can introduce significant stability issues. Small biases or deviations may accumulate across iterations, leading to divergence, oscillatory behavior, or collapse to degenerate fixed points [1]. To mitigate these effects, the recurrence is augmented with a gating mechanism that regulates how updates are incorporated at each step:

$$u_t = \mathcal{R}(h_t) \quad (3)$$

$$g_t = \sigma(W_z \cdot h_t + b_z) \quad (4)$$

$$h_{t+1} = g_t \odot u_t + (1 - g_t) \odot h_t \quad (5)$$

Here,  $g_t$  denotes a learned gate vector, computed via an element-wise sigmoid, that determines how much of the candidate update  $u_t$  is incorporated versus how much of the previous state  $h_t$  is preserved [1]. This gating operates at the level of individual dimensions, effectively acting as a selective damping mechanism: components of the representation that are already stable can be retained, while others continue to evolve [1]. In practice, this control is essential for maintaining stability across many iterations and preventing uncontrolled divergence at higher loop counts [1].

The recurrent core also incorporates multiple input injection strategies designed to preserve alignment with the original contextual representation produced by the Prelude [1]. Without such mechanisms, repeated application of  $\mathcal{R}$  may drift toward generic fixed points or amplify accumulated transformation artifacts, resulting in loss of input-specific information [1].

Several injection modes are supported [1]:

- *none*: performs no injection and offers minimal stability
- *residual*: adds a scaled projection of the initial state,  $h_t = h_t + \beta \cdot \text{project}(h_0)$ , providing a direct and effective anchor
- *concat\_project*: concatenates  $h_0$  with  $h_t$  and projects the result back to the model dimension
- *gate\_condition*: the gating function is conditioned on the initial state
- *attention\_memory*: introduces  $h_0$  as additional keys and values within the attention mechanism

Among these, the residual formulation offers a favorable balance of simplicity and empirical stability, and serves as the default configuration [1].

Additional measures are incorporated to maintain stability under repeated application of the recurrent block [1]. Spectral normalization is applied to weight matrices within the core to constrain the spectral radius and limit the risk of exponential growth across iterations [1]. During training, gradient clipping is used to prevent unstable updates arising from accumulated gradients [1].

The recurrent process is also subject to strict execution bounds [1]. A minimum number of iterations `min_loops` is always enforced to guarantee baseline reasoning depth, while a maximum limit `max_loops` caps computational extent [1]. These limits are treated as architectural constraints and are not overridden by the adaptive scheduler [1].

### 3.5 The Coda

The Coda follows the same structural design as the Prelude, consisting of  $L_c$  standard transformer blocks with pre-normalization, GQA-based self-attention, SwiGLU feed-forward layers, and residual connections [1]. Its role, however, differs from that of the earlier stage. After the recurrent core has completed its iterative refinement, the Coda acts as a final integration phase: it aggregates information across token positions, resolves dependencies that may remain partially formed, and transforms the iterated hidden state into a representation suitable for accurate next-token prediction [1].

This separation from the recurrent core is deliberate [1]. Iterative reasoning and output preparation impose different architectural demands. The recurrent core prioritizes stability and parameter sharing, while the Coda benefits from the flexibility and expressiveness of distinct layer parameters [1]. Combining these functions within a single stage would introduce trade-offs that weaken both [1].

### 3.6 Language Modeling Head

The final hidden state produced by the Coda is first normalized using RMS normalization and then passed through a linear projection that maps from the model dimension  $d_{\text{model}}$  to the vocabulary size [1]. This projection shares its weights with the token embedding matrix, so the same parameters are used both for input embedding and output projection [1]. This weight tying reduces the overall parameter count and is commonly associated with improved generalization [1]. Training is performed using the standard cross-entropy loss between the predicted token distribution and the ground-truth next token [1].

## 4 Training

### 4.1 Objective

Cognexis is trained using the standard autoregressive language modeling objective, in which the goal is to maximize the log-likelihood of each token conditioned on all preceding tokens [1]. Formally, for a sequence  $x = (x_1, \dots, x_T)$ :

$$\mathcal{L}(\theta) = - \sum_{t=1}^{T-1} \log P_{\theta}(x_{t+1} \mid x_1, \dots, x_t) \quad (6)$$

No specialized data or auxiliary objectives are introduced for the recurrent mechanism itself [1]. The recurrent core is trained solely through the standard language modeling signal, learning to refine hidden representations implicitly through next-token prediction [1]. This design treats

recurrence as a general-purpose mechanism for iterative refinement, rather than one optimized for specific task structures or externally imposed reasoning formats [1].

## 4.2 Loop Curriculum

Training a recurrent-depth model requires explicit handling of loop count during optimization [1]. A fixed-loop training regime leads to models that perform well only at that specific depth and degrade when evaluated at different iteration counts [1]. This behavior is incompatible with compute-adaptive inference, where depth is expected to vary across inputs [1].

To address this, a staged loop curriculum is employed, progressively expanding the model’s ability to operate effectively across a range of iteration counts [1].

**Phase 1 – Warm-up.** During the initial phase of training, a small number of recurrent iterations is used (e.g.,  $N = 2$ ) [1]. This stabilizes early learning by allowing the model to develop baseline language modeling behavior before relying heavily on iterative refinement [1]. In particular, the Prelude and Coda are able to form useful representations without depending on deep recurrence [1]. Beginning training with a large number of iterations can lead to instability, as the weight matrices in early stages are not yet well-conditioned and repeated application may amplify undesirable spectral properties [1].

**Phase 2 – Randomized Loops.** Following the warm-up phase, the loop count is varied across training batches, typically by sampling uniformly from the range  $[1, N_{\text{target}}]$  [1]. This phase is essential, as it exposes the model to a range of computational depths and encourages it to produce coherent outputs regardless of iteration count [1]. Dependence on a fixed number of loops is thereby avoided, and each iteration is implicitly trained to contribute meaningful refinement [1]. This variability also acts as a form of regularization, reducing overfitting to any particular depth and improving robustness across inference settings [1].

**Phase 3 – Depth Expansion.** In the final phase, the upper bound on loop count is progressively increased beyond the initial target range [1]. This stage encourages the model to operate effectively at greater depths than those encountered during earlier training, promoting generalization across iteration counts [1]. Sampling is biased toward these newly introduced higher loop values, while still retaining occasional low-loop examples to preserve performance at shallow depths and avoid degradation through catastrophic forgetting [1].

## 4.3 Optimization and Distributed Training

Optimization is performed using AdamW with standard hyperparameters ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.95$ , weight decay = 0.1) [1]. Training is conducted in mixed precision, with forward and backward passes executed in bfloat16 while maintaining master weights in full precision [1]. Memory usage is further reduced through gradient checkpointing, which recomputes intermediate activations during backpropagation instead of storing them, and through the use of fused CUDA kernels for attention and normalization operations [1].

For distributed training, Fully-Sharded Data Parallel (FSDP) or ZeRO (Rajbhandari et al., 2020) can be used to shard model parameters and optimizer states, with tensor parallelism applied within attention and feed-forward components for larger configurations [1]. A key consideration specific to recurrent-depth architectures arises from parameter sharing across iterations: since the same weights are reused at every loop, gradients accumulate contributions from all unrolled steps [1].



These contributions must be aggregated prior to the optimizer update [1]. This behavior is analogous to backpropagation through time in recurrent neural networks and introduces similar challenges, particularly with respect to controlling gradient scale and maintaining numerical stability [1].

#### 4.4 Stability and Regularization

Recurrent architectures introduce a stability concern that does not arise in standard transformers: repeated application of the same weight matrices can amplify spectral properties over iterations [1]. In particular, if the spectral radius exceeds unity, hidden-state norms may grow exponentially, leading to divergence [1]. To mitigate this, the spectral norm of weight matrices in the recurrent core is monitored during training, with optional enforcement of spectral normalization as a constraint [1]. RMS LayerNorm applied before each sub-layer provides per-iteration normalization, while residual scaling coefficients – initialized to small values – help stabilize early training dynamics [1].

Conventional regularization techniques remain applicable, including dropout within attention and feed-forward components, weight decay under AdamW, and optional stochastic depth through random skipping of recurrent iterations during training [1]. In addition, the training process includes explicit checks for numerical instability [1]. Activations within the recurrent core are continuously monitored, and the presence of non-finite values (NaN or infinity) triggers immediate termination with diagnostic reporting, preventing silent corruption of subsequent updates [1].

### 5 Inference and Adaptive Scheduling

#### 5.1 Prefill and Decode

Autoregressive generation follows the standard two-phase procedure used in decoder-only models [1]. In the prefill phase, the full input sequence is processed in parallel: the Prelude, all  $N$  iterations of the Recurrent Core, and the Coda are applied across the sequence, and the resulting key–value (KV) states are cached [1]. In the decode phase, tokens are generated sequentially, reusing cached KV states for previous positions while computing new states only for the current token [1].

The presence of recurrence introduces additional complexity in cache management [1]. In conventional transformers, each layer produces a single set of KV states that can be reused directly. In this setting, however, the hidden state evolves across recurrent iterations, and each iteration produces distinct attention states [1]. As a result, KV representations are depth-dependent rather than fixed [1].

A straightforward and reliable approach is to recompute the recurrent core’s KV states at each decode step, rather than attempting to reuse cached states across iterations [1]. While more efficient caching strategies may be possible, they require careful validation to ensure equivalence [1]. Incorrect reuse of KV states can lead to silent errors in generation, as inconsistencies may not produce explicit failures but instead degrade output quality [1].

#### 5.2 Adaptive Loop Scheduler

The Adaptive Loop Scheduler serves as the central runtime mechanism governing computation in Cognexis [1]. Its role is to determine, at inference time, how many recurrent iterations should be executed for a given input or, in more fine-grained settings, for individual tokens [1]. The objective is conceptually straightforward: allocate sufficient computation to achieve high-quality outputs while avoiding unnecessary iteration [1]. In practice, however, this decision is nontrivial, as the optimal

amount of computation depends on input complexity, task characteristics, and available resource constraints [1].

The scheduler supports four operating modes [1]:

- **Fixed:** Run exactly `loop_count` iterations. This is the baseline mode, used for training, debugging, and reproducible benchmarking. No adaptive logic is involved.
- **Rule-based:** Halt using hand-specified thresholds on observable quantities: hidden-state delta norms (how much the state changed), logit confidence (how peaked the output distribution is), and compute budget remaining.
- **Value-head:** Halt using a learned neural network that predicts the expected marginal improvement from an additional loop. When predicted gain drops below a threshold, the scheduler halts.
- **Hybrid:** Combine rule-based hard stops with value-head predictions. The rule-based component provides safety guardrails; the value head provides nuanced, learned judgment. This mode is recommended for production deployment.

At each iteration, the scheduler evaluates a set of signals reflecting the current state of computation [1]. These include the loop index, the norm of the change in hidden state relative to the previous iteration, measures of entropy and confidence in the current output distribution, the value head’s estimate of marginal gain, the remaining compute budget, and any active safety indicators [1]. These inputs are combined within a decision function that determines whether to continue iterating or halt [1].

Certain constraints are enforced as strict invariants [1]. A minimum number of iterations (`min_loops`) is always executed to ensure baseline reasoning depth, while a maximum limit (`max_loops`) prevents unbounded computation [1]. Execution must terminate when the available compute budget is exhausted, and any safety override takes precedence over learned behavior [1]. Within these constraints, a hybrid policy governs the halting decision through an ordered sequence: enforce hard bounds, satisfy minimum loop requirements, apply safety conditions, continue when the hidden-state change remains large, halt when output confidence is high and predicted gain is low, and otherwise defer to the value head’s gain-per-cost estimate [1].

### 5.3 Token-Wise Loop Allocation

In the most general scheduling regime, iteration counts may vary across token positions within a single sequence [1]. This reflects the uneven difficulty of prediction: some tokens (e.g., common function words) can be resolved early, while others – such as technical terms, ambiguous references, or intermediate reasoning steps – benefit from additional refinement [1]. Token-wise scheduling enables selective allocation of computation, concentrating effort where it is most impactful [1].

Implementation proceeds by associating each token position with an active state at every iteration [1]. The recurrent core updates only those positions that remain active, while halted positions retain their current hidden representations [1]. Halting is monotonic in the baseline formulation: once a token ceases to update, it remains fixed for all subsequent iterations [1]. This results in a deterministic execution pattern in which the set of active tokens decreases over time [1].

Attention over mixed-depth representations is handled with a simple and consistent rule [1]. Halted tokens preserve their most recent hidden states, while active tokens attend to all positions – both active and halted – using the latest available representations [1]. Consequently, tokens at

different iteration depths coexist within the same sequence [1]. Although this produces a mixed-depth state, the behavior remains well-defined and stable [1].

This form of scheduling introduces additional implementation and validation complexity relative to fixed or sequence-level adaptation [1]. As such, it is best approached after simpler scheduling strategies have been established and thoroughly tested [1].

## 5.4 The Value Head

The value head is a lightweight neural network that predicts the expected benefit of running one more recurrent iteration [1]. Its prediction target is the improvement in language modeling loss from loop  $t$  to loop  $t + 1$ :

$$\text{target\_gain}_t = \text{loss}_t - \text{loss}_{t+1} \tag{7}$$

A positive predicted gain indicates that an additional iteration is expected to improve performance, while a near-zero or negative value suggests that further computation is unlikely to be beneficial [1]. The value head is deliberately lightweight, consisting of RMS normalization applied to the hidden state, an optional embedding of the loop index, a small MLP with a bottleneck dimension typically between 64 and 512 units, and a final linear projection producing a scalar gain estimate [1]. Its computational cost is kept minimal relative to a full recurrent iteration, ensuring that the overhead of deciding whether to continue does not offset the savings from adaptive halting [1].

Accurate calibration is essential [1]. Overestimation of gain leads to unnecessary iterations and wasted computation, whereas underestimation results in premature halting and degraded output quality [1]. A practical training approach is to derive supervision directly from the base model: evaluate the model at multiple loop depths on validation data, record hidden-state summaries and corresponding losses, compute the empirical gain between successive iterations, and train the value head to predict these quantities [1]. Given the inherent noise in gain estimates, Huber loss is preferred over mean squared error for its robustness to outliers [1].

The value head is typically trained after the base model has converged, or introduced in a later calibration phase with a reduced learning rate [1]. Joint training from initialization can introduce interference between the primary language modeling objective and gain prediction [1]. Effective calibration is reflected in strong correlation between predicted and observed gains, and periodic recalibration is advisable when the underlying model is further fine-tuned [1].

## 6 Model Configurations

Four reference configurations are defined, spanning approximately three orders of magnitude in parameter count, from an 8B-scale model suitable for research settings to a 1.28T-scale model representative of frontier deployments [1]. These configurations are intended to characterize how the recurrent-depth approach behaves across scales [1]. In all cases, the recurrent core consists of a single shared block; increases in effective depth are achieved exclusively through additional iterations rather than by introducing new recurrent layers [1].

Larger models are trained with higher maximum loop counts, reflecting their greater representational capacity and ability to benefit from extended iterative refinement [1]. At inference time, however, the number of iterations remains a configurable parameter independent of model size [1]. A model trained with a given maximum loop count can be evaluated at higher or lower depths as needed, allowing computational effort to be adjusted to the requirements of the task [1].

Table 1: Cognexis Model Configurations

| Model          | Hidden<br>Size | Attn.<br>Heads | Prelude<br>( $L_p$ ) | Recurrent<br>( $L_r$ ) | Coda<br>( $L_c$ ) | Max Loops<br>Train | Params. |
|----------------|----------------|----------------|----------------------|------------------------|-------------------|--------------------|---------|
| Cognexis-8B    | 4096           | 32             | 8                    | 1                      | 8                 | 12                 | ~8B     |
| Cognexis-64B   | 8192           | 64             | 10                   | 1                      | 10                | 16                 | ~64B    |
| Cognexis-256B  | 12288          | 96             | 12                   | 1                      | 12                | 20                 | ~256B   |
| Cognexis-1.28T | 16384          | 128            | 16                   | 1                      | 16                | 24                 | ~1.28T  |

Note that effective depth at inference varies by loop count [1]. For example, Cognexis-8B with 12 loops has an effective depth of  $8 + 12 + 8 = 28$  block applications, comparable to a conventional 28-layer transformer [1]. At 1 loop, the same model has an effective depth of only 17 – a significantly lighter computation that may suffice for simple predictions [1].

## 7 Evaluation Framework

### 7.1 Conventional Metrics

Evaluation follows standard language modeling practice to ensure comparability with non-recurrent baselines [1]. Relevant metrics include perplexity, negative log-likelihood, exact match accuracy, multiple-choice accuracy, Pass@k for code generation, BLEU and ROUGE for generation quality, and human preference judgments for open-ended tasks [1].

For compute-adaptive models, reporting must include complete experimental context [1]. Each result should specify the dataset split, prompt format, decoding configuration (e.g., temperature, top- $p$ ), loop mode (fixed or adaptive, including iteration counts), and random seed [1]. Without this information, results cannot be reliably reproduced or meaningfully compared [1].

### 7.2 Depth Efficiency Index (DEI)

The Depth Efficiency Index measures how much performance improvement each additional unit of compute buys [1]. It is defined as:

$$\text{DEI}(N) = \frac{M(N) - M(N_0)}{C(N) - C(N_0)} \quad (8)$$

where  $M(N)$  is a performance metric (e.g., accuracy) at loop count  $N$ ,  $M(N_0)$  is the baseline performance at a reference loop count  $N_0$  (typically 1), and  $C(N)$  is the compute cost in FLOPs at loop count  $N$  [1]. In plain terms, DEI answers the question: “For each additional FLOP I spend on recurrence, how much better does my model get?” [1]. A high DEI indicates that extra loops are producing good returns; a low DEI suggests diminishing returns [1]. Plotting DEI as a function of  $N$  typically reveals a curve that peaks early and declines, providing a clear visual indicator of where the sweet spot lies for a given task [1].

### 7.3 Loop Saturation Point (LSP)

The Loop Saturation Point identifies the most efficient reasoning depth [1]. It is defined as the loop count that maximizes DEI:

$$\text{LSP} = \underset{N}{\operatorname{argmax}} \text{DEI}(N) \quad (9)$$

At the LSP, the model is getting the best return per unit of additional computation [1]. Beyond the LSP, each additional loop still helps (performance may still improve), but the rate of improvement per FLOP is declining [1]. Inference schedulers can use the LSP as a default loop budget: it represents the depth at which the model is operating most efficiently [1]. Importantly, LSP should be reported per task and per dataset, since different problems saturate at different depths [1]. A math reasoning benchmark may have an LSP of 10, while a simple text completion task may saturate at 2 or 3 [1].

#### 7.4 Overthinking Threshold (OT)

The Overthinking Threshold captures the point where additional computation actively hurts performance [1]:

$$\text{OT} = \min\{N : M(N + 1) < M(N)\} \quad (10)$$

This is the smallest loop count at which the model’s performance begins to decline with further iterations [1]. Overthinking occurs when extra iterations introduce noise, destabilize the hidden state, or cause the model to second-guess correct predictions [1]. It is a real and observed phenomenon in recurrent-depth models – the model can “think too much” and arrive at a worse answer than it had after fewer loops [1]. The adaptive scheduler should be configured to avoid exceeding OT [1]. Because OT is sensitive to individual inputs, it should be reported with confidence intervals across evaluation datasets [1].

#### 7.5 Depth Gain Ratio (DGR)

The Depth Gain Ratio provides a simple summary of how much recurrence helps overall [1]:

$$\text{DGR} = \frac{M(N_{\max}) - M(N_0)}{M(N_0)} \quad (11)$$

DGR compares the best performance achievable with full recurrence against a no-recurrence baseline [1]. A DGR of 0.15, for example, means that full recurrence improves performance by 15% over a single-pass baseline [1]. Higher DGR indicates that the recurrent architecture is providing substantial value and that the recurrence mechanism is not merely cosmetic [1]. DGR is most informative when compared across tasks: tasks with high DGR are the ones where compute-adaptive scheduling matters most [1].

#### 7.6 Scheduler Metrics

For adaptive scheduling modes, additional metrics are needed to assess the quality of the scheduling decisions themselves [1]. Key scheduler metrics include: average loops used, loop distribution histogram, halt reason distribution, quality at adaptive loops vs. same average fixed loop count, false-halt rate, false-continue rate, scheduler overhead, and budget violation count [1].

A critical evaluation principle: adaptive results must always be compared with fixed-depth baselines at equal total compute [1]. If the adaptive scheduler uses an average of 6 loops, its performance should be compared against a fixed-6-loop baseline [1]. Any performance difference is attributable to the scheduler’s ability to allocate loops non-uniformly – some inputs getting more, others getting less – rather than simply using more total computation [1].

## 8 Safety and Operational Considerations

The recurrent-depth design introduces a set of safety and operational considerations that are not present in conventional transformer architectures [1]. A primary concern is stability under iteration: excessive loop counts can lead to uncontrolled growth in hidden-state norms, convergence to degenerate fixed points, or oscillatory dynamics that produce unpredictable outputs [1]. Enforcing strict upper bounds on iteration count, along with continuous monitoring of activation magnitudes, is therefore a necessary architectural constraint [1]. Deployment systems require explicit safeguards that terminate recurrence if predefined stability thresholds are exceeded [1].

Adaptive scheduling further introduces the risk of unbounded resource consumption [1]. Certain inputs – whether adversarially constructed or inherently complex – may trigger excessive iteration, resulting in significantly higher computational cost than fixed-depth models [1]. To mitigate this, inference must be governed by explicit per-request compute budgets that cap total resource usage irrespective of scheduler decisions [1]. Without such constraints, the system remains vulnerable to denial-of-service scenarios that exploit its adaptive behavior [1].

Variability in iteration count also leads to variability in latency, which may expose timing side channels [1]. In contexts where this is a concern, controlled randomization of loop counts around the scheduler’s recommendation can reduce information leakage [1].

Safety mechanisms, including content filtering and refusal policies, must remain invariant across all loop configurations [1]. No choice of iteration count should weaken or bypass these safeguards [1]. Ensuring this requires systematic evaluation across the full range of depths, as safety behavior does not vary monotonically with additional computation [1]. In some cases, increased iteration may improve the model’s ability to identify and refuse unsafe requests; in others, it may amplify undesirable outputs [1]. Any loop regimes that degrade safety performance must be identified and restricted within the scheduler’s allowable range [1].

## 9 Limitations and Future Work

The Cognexis architecture inherits several limitations from the broader class of recurrent-depth models, alongside challenges specific to its design [1]. The most prominent concern is stability at high iteration counts [1]. Even with gating mechanisms, input injection, spectral normalization, and residual scaling, repeated transformation of hidden states remains inherently sensitive [1]. At sufficiently large loop counts, the system may exhibit divergent behavior, converge to uninformative fixed points, oscillate between attractors, or become overconfident in incorrect predictions [1]. These stabilization techniques are necessary but not universally sufficient, particularly across diverse inputs and scales [1]. Approaches based on contractive transformations or implicit fixed-point methods offer a potential path toward more robust behavior [1].

Training complexity is also significantly increased relative to standard decoder-only models [1]. The introduction of loop curricula, management of gradients across repeated iterations, and the need for stability monitoring expand both implementation complexity and the hyperparameter space [1]. Simpler training strategies that achieve robustness across depths without multi-phase curricula remain an open objective [1]. In parallel, adaptive scheduling introduces variability in inference latency, which complicates deployment in systems with strict service-level constraints [1]. Mechanisms that incorporate deadline awareness and allow controlled degradation of output quality may be required for practical use [1].

Additional open problems arise in several areas [1]. The generalization behavior of learned schedulers remains uncertain, particularly under distribution shift; miscalibration can lead to premature

halting or excessive computation [1]. More robust approaches may involve uncertainty-aware estimation or domain-adaptive recalibration [1]. The semantics of KV caching in the presence of recurrent updates are also more complex than in standard transformers, and incorrect reuse can introduce silent errors without explicit failure signals [1]. Extensions to multimodal settings raise further questions about how iterative computation should be distributed across heterogeneous inputs [1]. Integration with external tools introduces another layer of complexity, as scheduling must account for both internal iteration and external calls [1].

Finally, practical deployment requires consideration of hardware variability [1]. The optimal allocation of iterations depends not only on input characteristics but also on system-level factors such as compute throughput, memory bandwidth, and scheduling overhead [1]. Incorporating hardware-aware cost models into the scheduling process represents a natural direction for improving efficiency in real-world environments [1].

## 10 Conclusion

Cognexis reframes scaling in language models by separating reasoning depth from parameter count [1]. By reusing a shared recurrent transformer block, the architecture enables deeper effective computation – functionally comparable to much deeper networks – without increasing parameter storage or memory footprint [1]. The inclusion of a compute-adaptive interface, driven by an adaptive loop scheduler and a learned value head, allows inference-time allocation of computation to vary with input difficulty, assigning additional iterations where beneficial and terminating early when sufficient confidence is reached [1]. In this formulation, reasoning depth becomes a controllable runtime variable rather than a fixed architectural property [1].

The specification encompasses the full modeling stack, including tokenization, rotary positional encoding, the three-stage Prelude–Recurrent–Coda structure, stability mechanisms such as gating and input injection, and a multi-mode adaptive scheduling system [1]. It also defines a set of evaluation metrics tailored to iterative computation and provides reference configurations spanning several orders of magnitude in scale [1]. The level of detail is intended to support direct implementation, rather than serving as a high-level conceptual outline [1].

This perspective suggests a shift in how model capability is framed [1]. Rather than focusing solely on parameter count as the primary axis of improvement, attention moves toward how computation is allocated during inference [1]. If the amount of reasoning required varies across inputs, then architectures capable of adapting their depth accordingly offer both efficiency and potential gains in capability [1].

## References

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
- Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., & Kaiser, Ł. (2019). Universal Transformers. *International Conference on Learning Representations*.
- Bai, S., Kolter, J. Z., & Koltun, V. (2019). Deep Equilibrium Models. *Advances in Neural Information Processing Systems*, 32.

- Graves, A. (2016). Adaptive Computation Time for Recurrent Neural Networks. *arXiv preprint arXiv:1603.08983*.
- Geiping, J., McLeish, S. M., Jain, N., Kirchenbauer, J., Singh, S., Bartoldson, B. R., Kailkhura, B., Bhatele, A., & Goldstein, T. (2025). Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. *ES-FoMo III Workshop*.
- Lu, W., Yang, Y., Lee, K., Li, Y., & Liu, E. (2025). Latent Chain-of-Thought? Decoding the Depth-Recurrent Transformer. *arXiv preprint arXiv:2507.02199*.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568, 127063.
- Shazeer, N. (2020). GLU Variants Improve Transformer. *arXiv preprint arXiv:2002.05202*.
- Rajbhandari, S., Rasley, J., Ruwase, O., & He, Y. (2020). ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. *International Conference on High Performance Computing, Networking, Storage and Analysis*.
- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*.
- Hosseini, P. (2023). Quadratic Complexity of Transformers. *Towards Data Science*.
- Chowdhury, S. A., Varshney, N., & Baral, C. (2024). Adaptive Depth Transformers. *arXiv preprint*.